

VIXOR-API-001

VIXOR ENGINEERING BIBLE

إنجيل واجهات البرمجة

كل نقطة نهاية، نوع بيانات، وحدث في منظومة فيكسور

جدول المحتويات

نظرة عامة على طبقة API

واجهات Nitro REST (15 نقطة نهاية)

وظائف الخادم عبر TanStack (Server Functions)

نظام الأحداث (Domain Events)

المهام المجدولة (Cron Jobs)

التحقق والأمان

أنواع البيانات (DTOs)

خرائط API: الأمان ← الكود

نظرة عامة على طبقة API

تُبنى منظومة فيكسور فوق طبقتين متميزتين لواجهات البرمجة التطبيقية (API)، صُممت كل منهما لتلبية احتياجات مختلف البرمجية الشاملة. الطبقة الأولى هي **واجهات Nitro REST** التي تُدار عبر خادم Nitro المدمج في إطار العمل، وتوفر خمسة Endpoints (endpoint) تغطي مجالات الصحة والمقاييس والتدفق الذكي والاكتشاف والأسعار والتنبيهات المهية زمنياً وويب هوك تيار والتحقق من المحفظة. هذه الطبقة تتعامل مباشرة مع الطلبات الواردة من الخارج سواء كانت من خدمات المراقبة أو متصفحات أو بوابات تيليجرام.

الطبقة الثانية هي **وظائف الخادم عبر TanStack Start**، وهي وظائف تُنفَّذ على الخادم (Server Functions) تُستدعى الاستدعاء البعيد (RPC) من الواجهة الأمامية. تتميز هذه الطبقة بأنها تتوافق بشكل أعمق مع معمارية النطاقات (Architecture) المعتمدة في المشروع، حيث يملك كل نطاق وظيفي ملف ووظائف خاص به. تُغطي هذه الطبقة ستة عشر نطاقاً تشمل: المحلل والمساعد (Copilot) والتحليل والتداول والمستخدم وقائمة المراقبة وتتبع الإشارات والملاحظات والحلقة اليومية والمحفظة والتجارب واختبار الاسترجاع والمراجعة والتداول الورقي وإدارة الصفقات.

لماذا هذا التصميم المزدوج؟

واجهات Nitro مُحسَّنة للوصول الخارجي وخدمات الجهات الخارجية (مثل DexScreener وBinance وتيليجرام)، بينما وظائف TanStack تجربة المطور من خلال توفير استدعاءات مُنبثقة (type-safe RPC) تتجاوز حاجز الشبكة بشكل شفاف للواجهة الأمامية. هذا الفصل يضمن طبقة تؤدي دورها بأقصى كفاءة ممكنة دون تداخل في المسؤوليات.

بالإضافة إلى هاتين الطبقتين، تعتمد منظومة فيكسور على **نظام أحداث نطاق (Domain Events)** يُتيح التواصل غير المتزامن المختلفة. يُسجَّل كل حدث في جدول `domain_events` بقاعدة البيانات، مما يوفر سجلاً تاماً لكل نشاط يحدث داخل المنظومة. أيضاً **مهام مجدولة عبر Vercel Cron** لتنفيذ عمليات دورية مثل توليد الإشارات اليومية وفحص التنبيهات السعريّة.

واجهات Nitro REST (15 نقطة نهاية)

تُمثّل واجهات Nitro REST العمود الفقري للتواصل الخارجي في منظومة فيكسور. كل نقطة نهاية مُعرّفة في ملف مستقل `server/api/`، وتخضع لسلسلة من طبقات التحقق والأمان حسب حساسيتها. الجدول التالي يقدّم عرضاً شاملاً لجميع الخمس عشرة مع تفاصيلها الكاملة.

الطريقة	المسار	نوع المصادقة	الوصف	الملف
GET/HEAD	/api/health	CRON_SECRET / HEALTH_TOKEN	فحص صحة Redis و Supabase	server/api/health.ts
GET	/api/metrics	CRON_SECRET / HEALTH_TOKEN	مقاييس Prometheus	server/api/metrics.ts
POST	/api/copilot-stream	Bearer JWT	تدفق SSE للمساعد الذكي	api/copilot-stream.ts
GET	/api/discover	بدون (عام)	اكتشاف العملات عبر DexScreener	server/api/discover.ts
GET	/api/sol-price	بدون (عام)	سعر SOL/USDT من Binance	server/api/sol-price.ts

الطريقة	المسار	نوع المصادقة	الوصف	الملف
GET	/api/market-overview	بدون (عام)	بيانات أفضل الأسواق	api/market-overview.ts
GET/POST	/api/check-alerts	Admin / Vercel Cron	فحص التنبيهات السعريّة	api/check-alerts.ts
GET/POST	/api/generate-signals	Admin / Vercel Cron	توليد الإشارات اليومية	api/generate-signals.ts
POST	/api/telegram-webhook	Webhook Secret	ويب هوك بوت تيليجرام	api/telegram-webhook.ts
POST	/api/stars-webhook	Telegram Stars	ويب هوك دفع النجوم	api/stars-webhook.ts
POST	/api/migrate	Admin Key	تشغيل هجرات قاعدة البيانات	server/api/migrate.ts
POST	/api/p1-validate	Admin Key	التحقق من المرحلة الأولى	server/api/p1-validate.ts
GET/POST	/api/reanalysis-cron	Admin / Cron	إعادة تحليل الإشارات	api/reanalysis-cron.ts
GET POST	/api/wallet/connect	Bearer JWT	تحدي/التحقق من المحفظة	api/wallet/connect.ts
GET POST	/api/wallet/session	Bearer JWT	جلسات المحفظة	api/wallet/session.ts

تتوزع نقاط النهاية هذه بين ثلاث فئات أمنية: نقاط عامّة مفتوحة (`/api/discover`) و `api-price` و `api/market-overview` (لا تتطلب أي مصادقة، ونقاط محمية بـمفتاح مسؤول أو رمز كرون (`/api/health`) و `api/price-tricks` و `/api/check-alerts` و `/api/generate-signals` و `/api/migrate` و `api/p1-validate` و `api/reanalysis-cron`), ونقاط تتطلب مصادقة مستخدم عبر JWT (`api/copilot-stream` و `api/wallet/connect` و `api/wallet/session`). نقطة `/api/telegram-webhook` تتطلب سرّيّة ويب هوك خاصّة بينما `api/stars-webhook` تعتمد على تحقّق النجوم من تيليجرام.

ملاحظة على التقييد (Rate Limiting)

نقاط النهاية العامة تخضع لتقييد صارم لمنع إساءة الاستخدام. نقاط المصادقة تحدّد من عدد الطلبات لكل مستخدم. نقاط المسؤول لا تُقيّد لحماية بمفاتيح مشفّرة.

SEC - 03

وظائف الخادم عبر TanStack (Server Functions)

وظائف الخادم في TanStack Start هي الآلية الأساسية للتواصل بين الواجهة الأمامية والخادم في منظومة فيكسور. تُعرّف هذه الوظائف بـ `createServerFn` وتُستدعى من المتصفح كما لو كانت دوال محلية، لكنها تُنفَّذ بالكامل على الخادم. هذا النمط يُلغي الحاجة لطلبات HTTP يدوية ويوفّر سلامة الأنواع (type safety) من الطرف إلى الطرف. يغطي الجدول التالي جميع النطاقات السالكة عشر

النطاق	الملف	الوظائف
moxi	src/domains/moxi/functions.ts	askMoxi, updateMoxiPersona, getMoxiPersonaFn, getMoxiInsights
copilot	src/domains/copilot/functions.ts	askCopilot, إدارة المحادثات
analysis	src/domains/analysis/functions.ts	تشغيل, جلب, قائمة, حذف التحليلات
trading	src/domains/trading/functions.ts	إنشاء, جلب, تحديث التنبيهات ووظائف التداول
user	src/domains/user/functions.ts	تسجيل الدخول عبر تيليغرام, تحديث الملف الشخصي
watchlist	src/domains/watchlist/functions.ts	إضافة, جلب, حذف عناصر قائمة المراقبة
signal-tracking	src/domains/signal-tracking/functions.ts	تتبع الإشارات CRUD
notes	src/domains/notes/functions.ts	ملاحظات التداول CRUD
daily-loop	src/domains/daily-loop/functions.ts	الحلقة اليومية CRUD
wallet	src/domains/wallet/functions.ts	ربط المحفظة بالحساب
discovery	src/domains/discovery/functions.ts	scanDiscovery, searchTokens
experiment	src/domains/experiment/functions.ts	إدارة التجارب CRUD
backtest	src/domains/backtest/functions.ts	تنفيذ اختبار الاسترجاع
arbitrage	src/domains/arbitrage/functions.ts	فحص فرص المراجعة
paper-trading	src/domains/paper-trading/	تنفيذ التداول الورقي
trades	src/domains/trades/functions.ts	إدارة الصفقات CRUD

كل نطاق وظيفي يملك ملف `functions.ts` خاصاً يُؤطر جميع الوظائف المتعلقة به. هذا التنظيم يجعل النظام قابلاً للتسهيل، حيث يمكن إضافة نطاق جديد بإنشاء مجلد وملف وظائف دون التأثير على النطاقات الأخرى. جميع وظائف القراءة تستخدم `POST`. تتمّ جميع الوظائف التي تتطلب مصادقة عبر وسيط `AuthenticateRequest`. يتحقق من رمز JWT المُرسَل في ترويسة `Authorization`.

ميزة RPC المُنبثق

الواجهة الأمامية تستدعي هذه الوظائف مباشرة دون كتابة عناوين URL أو إدارة طلبات TypeScript. HTTP يتأكد من تطابق أنواع المدخلات والمخرجات بين الخادم والعميل في وقت الترجمة، مما يُقلّل الأخطاء البرمجية بشكل كبير.

SEC - 04

نظام الأحداث (Domain Events)

يعتمد نظام فيكسور على معمارية أحداث نطاقية (Domain Events) تُتيح التواصل غير المتزامن والمُفكّك بين المكونات المختلفة. يُمثّل تغييراً مهماً في حالة النظام ويحمل حمولة (payload) مُحددة تصف ما حدث، يُسجّل كل حدث في جدول `in_events` بيانات Supabase، مما يوفر سجلاً تاماً قابلاً للتدقيق لكل نشاط. يُنفذ النظام عبر كائن `VixorEvents` المركزي الذي يُصدِر الأ

أحداث التحليل والإشارات

الحدث	الحمولة (Payload)
analysis.created	analysisId, pair, timeframe, userId, recommendation, confidence
analysis.failed	pair, userId, error
signal.generated	pair, timeframe, recommendation, confidence, signalDate
signal.expired	pair, signalDate
signal.tracking.created	trackingId, userId, pair, direction, entryPrice, stopLoss
signal.tp_hit	trackingId, userId, pair, direction, prices
signal.sl_hit	trackingId, userId, pair, direction, prices

أحداث التنبيهات والتداول

الحدث	الحمولة (Payload)
alert.triggered	alertId, userId, pair, condition, targetPrice
alert.created	alertId, userId, pair, condition, targetPrice
trade.opened	tradeId, userId, pair, prices
trade.closed	tradeId, userId, pair, prices, pnl
trade.updated	tradeId, userId, pair, prices, pnl

أحداث المستخدم والحلقة اليومية

الحدث	الحمولة (Payload)
user.telegram.linked	userId, telegramId
user.premium.subscribed	userId, plan, expiresAt
user.points.credited	userId, amount, reason
dailyloop.morning-prep.completed	userId, date
dailyloop.eod-review.completed	userId, date
journal.created	noteId, userId, pair, mood
journal.updated	noteId, userId, pair, mood
notification.created	notificationId, userId, type, title
copilot.message.sent	conversationId, userId, agentId
copilot.action.executed	conversationId, userId, agentId, action
watchlist.item.added	userId, pair, watchlistId

يضمن هذا التصميم أن أي مكون في النظام يمكنه الاستجابة للأحداث دون معرفة المكون الذي أطلقها. مثلاً، يمكن لوحدة الإشعارات إرسال إشارة `signal.tp_hit` وإرسال إشعار للمستخدم، بينما يمكن لوحدة التحليلات تحديث إحصائياتها بناءً على نفس الحدث يُبسّط إضافة ميزات جديدة دون تعديل الكود الموجود.

SEC - 05

المهام المجدولة (Cron Jobs)

تعتمد منظومة فيكسور على **Vercel Cron Jobs** لتنفيذ مهام دورية تضمن استمرارية تشغيل المنظومة وتحديث بياناتها بشكل آلي. المهام في ملف `vercel.json` ضمن قسم `cron` ، وتُستدعى نقاط النهاية المحددة وفقاً للجدول الزمني المُهيأ. كل مهمة `CRON_SECRET` يتم التحقق منه قبل تنفيذ أي منطق، مما يمنع الاستدعاءات غير المصرح بها من جهات خارجية.

جدول المهام المجدولة

الجدول الزمني	التوقيت (UTC)	نقطة النهاية	الوصف
<code>* * * * *</code>	00:00 منتصف الليل	<code>/api/generate-signals</code>	توليد إشارات التداول اليومية لجميع أزواج العملات
<code>* * * * 30</code>	00:30 بعد منتصف الليل	<code>/api/check-alerts</code>	فحص جميع التنبيهات السعريّة النشطة ومقارنتها بالحالية

تعمل مهمة توليد الإشارات عند منتصف الليل بتوقيت UTC بالتوازي مع افتتاح الأسواق اليومية الجديدة. تقوم هذه المهمة بمسح العملات المُتتَبَعة وتشغيل محرك التحليل الفني لكل زوج، ثم تُخزّن الإشارات الناتجة في جدول `signals` مع تحديد مستوي الوقف والأهداف. بعد ثلاثين دقيقة، تُنفَّذ مهمة فحص التنبيهات التي تُقارن الأسعار اللحظية المسترجعة من Binance و Per بالشروط المُعَيَّنة في كل تنبيه، وتُطلق أحداث `alert.triggered` عند تحقق أي شرط.

آلية التحقق من المهام

كل طلب من Vercel Cron يتضمن ترويسة `Authorization: Bearer {CRON_SECRET}`. يتم التحقق من هذه الترويسة عبر دالة `validateAdminKey` في بداية كل نقطة نهاية مُهيأة زمنياً. في حال فشل التحقق، يُردّ خطأ `401 Unauthorized` دون تنفيذ أي منطق. يضمن أن المهام لا تُنفَّذ إلا بواسطة منصة Vercel نفسها.

بالإضافة إلى هاتين المهمتين الرئيسيتين، توجد نقطة نهاية `/api/reanalysis-cron` التي تُعاد تحليل الإشارات السابقة لتحديثها بناءً على التطوّر اللاحق للأسعار. يمكن تفعيل هذه المهمة بجدول زمني إضافي أو استدعاؤها يدوياً من لوحة المسؤول. هذا النمط المنظومة تعمل بشكل مستمر دون تدخل بشري مباشر، مع الحفاظ على أعلى معايير الأمان والموثوقية.

SEC - 06

التحقق والأمان

تُطبّق في منظومة فيكسور ثلاث طبقات تحقق رئيسية تغطي جميع نقاط الوصول. كل طبقة مُصمّمة لسيناريو محدد وتُنفَّذ متطلبات الأمان لكل نقطة نهاية.

validateAdminKey

تتحقق من مفتاح المسؤول المُرسَل في ترويسة `Authorization: Bearer <token>` أو معامل `key`. تُستخدم لحماية نقاط الإدارة مثل `/api/p1-validate` ونقاط الكرون. المفتاح يُخزّن في بيئة `CRON_SECRET` أو `ADMIN_KEY`.

authenticateRequest

الطبقة الأساسية لمصادقة المستخدمين. تتحقق من رمز JWT في ترويسة `Authorization: Bearer <token>`، تستخرج معرف المستخدم (`userId`) وتُمرّره إلى معالج الطلب. تُستخدم في جميع وظائف TanStack Server Functions التي تتطلب صلاحية مستخدم، وفي نقاط Nitro مثل `/api/copilot-stream` و `*/api/wallet`.

تُولَّد أنواع TypeScript تلقائياً من مخطط Supabase عبر أداة `supabase gen types`. هذا يضمن تطابق تام بين هيكل قاعدة البيانات والبيانات المستخدمة في الكود، ويُلغى الحاجة لتحديث يدوي عند تغيير المخطط.

خرائط API: الأمان ← الكود

يُقدِّم هذا الفصل خريطة مرجعية شاملة تربط كل نقطة نهاية بطبقة الأمان التي تحميها ومسار الملف في الكود المصدري. تُساعد المطورين ومُدققي الأمان في فهم نمط الحماية المُطبَّق بسرعة وتحديد أي فجوات محتملة.

نقاط محمية بـ CRON_SECRET / HEALTH_TOKEN

نقطة النهاية	آلية التحقق	مسار الملف
GET/HEAD /api/health	validateAdminKey	server/api/health.ts
GET /api/metrics	validateAdminKey	server/api/metrics.ts
GET/POST /api/check-alerts	validateAdminKey	server/api/check-alerts.ts
GET/POST /api/generate-signals	validateAdminKey	server/api/generate-signals.ts
GET/POST /api/reanalysis-cron	validateAdminKey	server/api/reanalysis-cron.ts

نقاط محمية بـ Admin Key

نقطة النهاية	آلية التحقق	مسار الملف
POST /api/migrate	validateAdminKey	server/api/migrate.ts
POST /api/p1-validate	validateAdminKey	server/api/p1-validate.ts

نقاط محمية بـ Bearer JWT

نقطة النهاية	آلية التحقق	مسار الملف
POST /api/copilot-stream	authenticateRequest	server/api/copilot-stream.ts
GET POST /api/wallet/connect	authenticateRequest	server/api/wallet/connect.ts
GET POST /api/wallet/session	authenticateRequest	server/api/wallet/session.ts

نقاط محمية بـ Webhook Secret

نقطة النهاية	آلية التحقق	مسار الملف
POST /api/telegram-webhook	Webhook Secret (Telegram)	server/api/telegram-webhook.ts
POST /api/stars-webhook	Telegram Stars Validation	server/api/stars-webhook.ts

نقاط عامة (بدون مصادقة)

مسار الملف	الحماية	نقطة النهاية
server/api/discover.ts	Rate Limiting فقط	GET /api/discover
server/api/sol-price.ts	Rate Limiting فقط	GET /api/sol-price
server/api/market-overview.ts	Rate Limiting فقط	GET /api/market-overview

جميع وظائف TanStack Server Functions الستة عشر المذكورة في الفصل الثالث تخضع لـ `authenticateRequest` بدلا من `validateRequest` عند تعريفها بـ `.validator()`. النقاط العامة محمية بتقييد الطلبات فقط عبر Redis لمنع إساءة الاستخدام دون عرقلة الودائع. يُنصح بمراجعة هذه الخريطة عند إضافة أي نقطة نهاية جديدة لضمان تطبيق طبقة الأمان المناسبة.

وثيقة رسمية لمنظومة فيكسور الهندسية

VIXOR — AI-Powered Trading Terminal